DTree Public API Reference

Overview

The `DTree` class represents a symbolic directed tree with a unique root and internal support for dual projections (Q1 and Q2). Each node is identified by a unique string. Edges are added incrementally, and projections are updated automatically. Structural operations preserve the tree's topological validity.

Constructor

Constructor – DTree(string id)
`Create a new tree with a given identifier`

- Initializes a new directed tree with the given identifier.
- The root is determined implicitly as the source of the first inserted edge.
- Q1 and Q2 projections are initialized automatically.

Core Methods

Insert – void Insert(string parent, string child)
`Insert a new edge into the tree`

- Adds a directed edge from `parent` to `child`.
- The `parent` must already exist in the tree.
- The `child` must not already exist.
- Updates Q1 and Q2 projections accordingly.

Contains – bool Contains(string node)
`Check if a node exists`

- Returns `true` if the specified node is present in the tree.
- Otherwise returns `false`.

GetParent – string GetParent(string node)
`Return the parent of a node`

- Returns the parent of `node`.
- If `node` is the root, returns `null`.
- Throws an error if the node is not found.

Structural Reconfiguration

CutNPaste – void CutNPaste(string u1, string u2)
`Detach a subtree from its root and attach it to another`

- Extracts the subtree rooted at `u2` from the tree rooted at `u1`.
- The subtree is detached from its original parent and reattached under a new one.
- Ensures that no orphaned structure remains in the source tree.
- Both source and destination trees must be structurally valid after the operation.

Delete – void Delete(string node)
`Remove a node and reattach its children, if any, to its parent`

- Deletes `node` from the tree.
- If `node` has children, they are reattached to `RootOf(node)`.
- Behaves as a shorthand for `CutNPaste(node, RootOf(node))`.
- If `node` is the root, the operation is disallowed.

Accessors

Q1 – DList Q1
`Return the Q1 projection`

- Returns a copy of the Q1 projection (left-to-right traversal).
- The projection reflects the current visible tree state.

Q2 – DList Q2
`Return the Q2 projection`

- Returns a copy of the Q2 projection (right-to-left traversal).
- Q2 is the sibling-reversed counterpart of Q1.

Source – string Source
`Return the symbolic source node`

- Returns the symbolic start marker of all projections.
- This node is **not** part of the tree structure.
- It conceptually acts as the left bracket delimiting Q1 and Q2.

Sink – string Sink
`Return the symbolic sink node`

- Returns the symbolic end marker of all projections.
- This node is **not** part of the tree structure.
- It conceptually acts as the right bracket delimiting Q1 and Q2.

Query Methods

Reachability

CanReach – bool CanReach(string u, string v)
`Check if one node can reach another`

- Returns `true` if there is a directed path from node `u` to node `v`.
- The check is structural and respects tree hierarchy.

CanNotReach – bool CanNotReach(string u, string v)
`Check if one node cannot reach another`

- Returns `true` if there is no directed path from node `u` to node `v`.
- Logical negation of `CanReach(u, v)`.

Ancestry and Structure

IsRoot – bool IsRoot(string node)
`Check if a node is the root`

- Returns `true` if `node` is the root of the tree.

IsLeaf – bool IsLeaf(string node)
`Check if a node has no children`

- Returns `true` if `node` has no outgoing edges.

IsAncestor – bool IsAncestor(string u, string v)
`Check if one node is an ancestor of another`

- Returns `true` if node `u` lies on the path from the root to node `v`.

IsDescendant – bool IsDescendant(string u, string v)
`Check if one node is a descendant of another`

- Returns `true` if node `u` lies below node `v` in the tree.
- Equivalent to `IsAncestor(v, u)`.

SameSubtree – bool SameSubtree(string u, string v)
`Check if two nodes belong to the same tree`

- Returns `true` if both nodes share the same root.
- Useful for validating `CutNPaste` targets.

SubtreeSize – int SubtreeSize(string node)
`Return the number of nodes in a subtree`

- Returns the size (inclusive) of the subtree rooted at `node`.
- Includes the node itself and all its descendants.

Depth – int Depth(string node)
`Return the depth of a node`

- Returns the distance from the root to `node`.
- The root has depth 0.

PathToRoot – List¡string¿ PathToRoot(string node)
`Return the list of nodes from a node to the root`

- Returns the sequence of nodes on the path from `node` to the root.
- The last entry in the list is the root itself.

RootToPath – List¡string¿ RootToPath(string node)
`Return the list of nodes from the root to a node`

- Returns the sequence of nodes on the path from the root to `node`.
- The first entry in the list is the root itself.